

Git Gud (at Git)

This document shall serve as both kickstart and quick reference for beginners. Though even experienced users may learn something new.

For a more complete instructions have a look at [Pro Git](#). You can access the book either as HTML document, PDF or EPUB. Change the websites language for localised versions of the book.

Table of contents

- [Git Gud \(at Git\)](#)
 - [Table of contents](#)
 - [Terminology](#)
 - [Git & Github](#)
 - [Git help](#)
 - [Configuration](#)
 - [Creating a repository](#)
 - [The concept of commits](#)
 - [Creating a commit](#)
 - [.gitignore files](#)
 - [Branches](#)
 - [Creating/switching a branch](#)
 - [Merging/rebasing a branch](#)
 - [Merge or rebase?](#)
 - [Merging](#)
 - [Rebasing](#)
 - [Resolving conflicts](#)
 - [History inspection](#)
 - [History rewriting](#)
 - [Troubleshooting](#)
 - [I lost a commit!](#)
 - [I tried to pull, but got a merge conflict?!](#)
 - [Further reading](#)
 - [Glossary](#)

Terminology

Command examples may have arguments enclosed with angled brackets (<>). That means the user needs to substitute the argument with their value. The text inside the angled brackets describes the expected value.

Additionally an argument may be enclosed by brackets (`[]`). That means the argument is optional.

Git & Github

Some might be confused about Git and Github. They have similar names, with Github just adding a 'hub'.

- **Git** is a version control system, that lets you save, analyse and edit the history of files. It can be used locally, or in combination with a server. The latter allows for collaboration with a team and can act as backup. Git is licensed under GPLv2.
- **Github** is a frontend for Git, which adds functions like a webserver, access control, pull requests or issues to name a few. It is not a usable service on its own. Github is proprietary and owned by Microsoft since 2018.

Git help

When unsure about a Git (sub)command append `--help`. This opens the most specific manual available for your command.

Example:

```
1 # show help for the 'config' subcommand
2 git config --help
3
4 # show help for the 'remote add' subcommand
5 git remote add --help
```

Configuration

Before starting to use Git it is recommended to configure the user name and email at the very least. Otherwise, Git will ask you at the first action which requires an identity, most probably `push`.

Git looks for a file `~/.gitconfig` (`~` = users home directory). You can either (create and) edit the file directly, or use `git config`. See the example below for how to configure the user name and mail.

```
1 [user]
2     name = student
3     email = student@students.zhaw.ch
```

Alternatively use the following commands to let Git write the configuration file.

```
1 git config set user.name <you-display-name>
```

```
2 git config set user.email <your-address@domain.global-domain>
```

For Linux users keeping their configuration files in a repository, distributing them with symbolic links, the Git configuration can include files. So the user and email can be stored in a separate file.

```
1 [include]
2     path = /path/to/include/file
```

Sensible options to set are the editor used for commit messages, the default branch name and whether to merge or rebase. The value of editor is the launch command of the desired editor. Below is an excerpt from my configuration as an example.

```
1 [core]
2     editor = nvim
3
4 [init]
5     defaultBranch = main
6
7 [pull]
8     rebase = true
```

Creating a repository

When working with a Git frontend like Github there are two ways to start a repository:

- **Local:** Use `git init` to initialise the current directory as a Git repository. This will create a directory `.git` which contains the local information, e.g. history, commit hashes, hooks, etc. If you want to synchronise a local repository with a server, follow the steps for creating a remote repository and use `git remote add` to add a remote and its URL (default remote name is `origin`).
- **Remote:** Open the frontend of choice and create a new repository. On Github there is usually a green button in the top right corner. Choose a name, description and licence. Now get the URL to clone the repository locally, or set it as remote in an existing one. On Github click the green `Code` button and choose either **https** or **ssh** (recommended, but requires additional setup).

See examples for the Git commands mentioned in this section below.

```
1 # initialise directory as git repository
2 # use '.' as path to initialise current directory
3 git init [<path/to/directory>]
```

```
4
5 # add remote to local repository under name 'origin'
6 git remote add origin <url-to-remote-repository>
7
8 # change URL, for example to switch to ssh after cloning
9 # a repository over https
10 git remote set-url <remote-name> <remote-url>
11
12 # clone a remote repository
13 git clone <repository-url>
```

Mind, that a remote is not strictly necessary, because the local repository contains the full information for version control. A remote is useful as a backup of your information and to work with other contributors.

Choosing a licence is important, if only to deny liability for your code.

I recommend to setup a pair of ssh keys and use ssh connections to interact with the Git server. It is more secure and usually more convenient too.

The concept of commits

Before looking at how to add or change commits, it is prudent to get an understanding of what a commit is.

A Git history is a linked list of changes, called commits. Each commit has one (or in case of merge commits more) parents and an arbitrary number of children. A commit is a delta relative to its parent(s) and does **not** directly contain the current state.

Creating a commit

Use `git add <file-or-directory>` to stage changes made to a file or directory (a directory means all contained files, recursively). List the changed files in your working tree with `git status` and show the differences with `git diff [<path>]`.

With `git commit` you can commit the staged changes. This will open the configured editor. Enter your commit message at the very top, save and close the file.

```
2 # Please enter the commit message for your changes. Lines
   starting
3 # with '#' will be ignored, and an empty message aborts the
   commit.
4 #
5 # On branch main
6 # Your branch is up to date with 'origin/main'.
7 #
8 # Changes to be committed:
9 #   modified:   git-introduction/readme.md
10 #
```

You now created a commit, but it is only local, use `git push` to upload it to the remote repository (This may fail due to conflicts, see [Resolving conflicts](#)).

Instead of opening an editor you can also commit with `git commit -m <commit-message>` but I prefer opening an editor, because it gives me another chance to spot missing changes, or changes I did not want to add to the commit.

It is sensible to make small commits with coherent scope.
When adding a source file it is reasonable to commit the change to the Makefile too.
However committing two code changes for distinct functionality at once is frowned upon.
The reason is, that when searching the history for a commit, it gets really difficult to find the change you are looking for, when commits contain a lot of changes.

.gitignore files

Committing files generated by the project is a **crime**, punishable with a sentence to read the entire Git documentation.

Manually ignoring the generated files would be quite tedious, thus Git has a mechanism to automatically ignore changes matching a user defined pattern. It is very simple, just add a file `.gitignore` to any directory in your repository. Such an ignore file is applied for the directory they reside in and all lower levels. An ignore file has no effect on its parent or sibling directories.

Here is an example of a `.gitignore`:

```
1 # ignore build directory (and all its contents)
2 build/
3
4 # ignore all pdf
5 *.pdf
6 # do not ignore specific pdf
7 !readme.pdf
```

Branches

A branch is essentially a named reference to a list of commits, originating from a commit on the main branch. Here is an ASCII graphic to try and explain the concept. Keep in mind, that commits (represented by capital letters) contain only the differences to their parents.

```
1 A -- B -- D ----- G (main)
2      \
3      C -- E -- F (branch)
```

Branches are a good tool to work on a project with several contributors without solving conflicts on every commit. Or to quickly try an idea without the risk to poison the development branch.

Imagine the a scenario, where you work with a colleague on a project. Your colleague works one part and you on another but both of you have to make changes to the glue logic inbetween. The one who commits second will have to solve conflicts. This still applies when both of you work on separate branches, but only once, when you rebase or merge your branch back to mainline. While working on your branch you never need to solve conflicts.

Another use for branches is to develop several features concurrently. Working with branches you jump between the features without having to worry about one to interfere with another. Also you can have a branch reviewed while working on a different feature.

Creating/switching a branch

There are at least two methods to create a branch. Here are the two that come to mind first:

```
1 # create a branch,
2 # but do not switch to the created branch
3 git branch <branch-name>
4
5 # create a branch and switch to it
6 git checkout -b <branch-name>
```

If your working tree is clean, you can switch between branches with the following commands:

```
1 # change to branch (or commit)
2 # (does not create branches or commits)
3 git checkout <branch-name/commit-hash>
4
```

```
5 # change to branch
6 git switch <branch-name>
```

Merging/rebasing a branch

When a feature is implemented and tested, it is usually to be integrated into the main branch.

Git provides two ways to achieve this, merging and rebasing.

The following subchapter explains the two philosophies.

Merge or rebase?

My quick answer is: Does not matter, but use only one strategy per repository.

This chapter will give a quick explanation of what *merging* and *rebasing* means and at the end I explain my opinion.

Merging means creating a commit with two parents. When merging, all conflicts are resolved in one merge commit.

Here is some ASCII art to visualise this concept:

```
1 # merge
2
3 A -- C (main)
4 \
5  B -- D -- E (feature branch)
6
7 | | | |
8 v v v v
9
10 A -- C ----- F (main)
11 \                   /
12  B -- D -- E (feature branch)
```

Rebasing means to recalculate the deltas from a new origin. This way one can just add (fast forward) the commits of the feature branch on top of the main branch. When rebasing, conflicts are resolved in the commit of the feature branch, that introduces the conflict.

Here is some ASCII art to visualise this concept:

```
1 # rebase
2
3 A -- C (main)
```

```

4  \
5   B -- D -- E (feature branch)
6
7  |   |   |   |
8  v   v   v   v
9
10 A -- C (main)
11     \
12      B' -- D' -- E' (feature branch)
13
14 # rebased feature-branch on top of main
15
16 |   |   |   |
17 v   v   v   v
18
19 A -- C -- B' -- D' -- E' (main)
20
21 # rebased main on top of rebased feature-branch
22 # (no conflicts -> fast forward)

```

Conclusion both strategies have advantages and disadvantages:

- *Merging* has the advantage of keeping more information.
- *Rebasing* has the advantage of better readability of the history.
- Some actions for history cleanup require the interactive *rebase* tool, which may be a *rebasing* argument for some.

I personally have a preference for *rebasing* due to the cleaner history, but find it perfectly reasonable to use *merging*.

Merging

When merging, checkout the destination branch and merge the feature branch into it. You will get one merge commit, that solves all conflicts. Both branches stay in your history and are displayed side to side in the graph view.

Here I explain the procedure of a merge, below you find the commands used.

- 1) First synchronise the local with the remote repository.
- 2) Checkout the **destination** branch.
- 3) Merge the feature branch into your current (destination) branch.
- 4) This may cause conflicts.
 - 1) Get a list of files containing conflicts with `git status`.
 - 2) Resolve conflicts (see [Resolving conflicts](#))
 - 3) Continue with `git merge --continue`, choose a message for the merge commit, save and close.
- 5) Synchronise the remote with `git push`.

Rebasing

When rebasing, checkout the feature branch and rebase it on the main branch. This way all conflicts are resolved on the feature branch and the commits can be fast forwarded to the main branch. Rebasing requires a force push.

Many projects prohibit force pushes on the main branch, so rebasing the main branch on top of the feature branch is not even an option.

Be very careful with force pushes.
Always substitute `--force` with `--force-with-lease`. This at least disables one to overwrite commits on the remote, that are not known locally.

Here I explain the procedure of a rebase, below you find the commands used.

- 1) First synchronise the local with the remote repository.
- 2) Checkout the **feature** branch.
- 3) Rebase the feature branch to the destination branch
- 4) This might cause conflicts.
 - 1) Get a list of files containing conflicts with `git status`.
 - 2) Resolve conflicts (see [Resolving conflicts](#)).
 - 3) Continue the rebase with `git rebase --continue`.
 - 4) Repeat until all conflicts are solved. When rebasing, conflicts are solved in the commit, which introduces them. Thus one may have to repeat these steps as many times, as there are commits to rebase.
- 5) Force push your rebased branch (with `--force-with-lease`).
- 6) Check out the destination branch.
- 7) Rebase destination branch to feature-branch.
- 8) Push rebased destination branch (this does not require force).

```
1 # fetch the newest state from the remote
2 git fetch --all
3
4 # switch branch
5 git checkout <feature-branch>
6
7 # rebase current branch onto <branch>
8 git rebase <branch>
9
10 # show files containing conflicts during a rebase
11 git status
12
13 # stage solved conflicts
14 git add <file/directory>
15
```

```
16 # continue rebase after conflicts are solved
17 git rebase --continue
18
19 # push rebased commits to feature branch
20 #
21 # NEVER USE '--force', SUBSTITUE WITH '--force-with-lease'!
22 #
23 # be very careful with force pushes '--force-with-lease' is
24 # still a potential foot gun!
25 git push --force-with-lease
```

Resolving conflicts

For resolving conflicts you can either use a graphical mergetool, or decide to do it manually (in the file itself). I do not use any GUI for Git, so can not give you any recommendations, or instructions for how to use them. I will just describe the manual approach, it is no more difficult really.

Git will insert searchable sequences around the conflicts and label both versions with the source of the change. See the example below.

```
1 <<<<<<< HEAD
2 hello world
3 =====
4 blargh
5 >>>>>>> f837762 (b)
```

To solve a conflict, decide which parts you want to keep and delete the rest of the changes encased by Gits markers. If you want or need to you can also mix the changes of both commits. Then delete the markers and save the file. Stage the file and continue the merge or rebase with `git merge --continue` or `git rebase --continue` respectively.

History inspection

You can inspect the history with `git log`.

Use `-n <number>` or just `-<number>` to show only entries.

Use `--all` to show commits newer than HEAD, other branches and remote references.

Use `--oneline` to get a compact view. Combine it with `--graph` to display the commit tree as ASCII art.

Use `git show <commit-hash/commit-reference>` to get the diff of the commit.

History rewriting

This is an advanced topic not recommended for beginners. I suggest to get used to work with Git first and come back once the desire to clean up the history rises.

The primary tool for history rewriting is the interactive rebase (`git rebase -i`). It provides a multitude of functions, reorder, reword and squash to name just a few.

Using `git rebase -i <commit-reference/commit-hash>` will open a list of all commits following the provided commit. In the development repository for CT for example I used the command `git rebase -i HEAD~3` to get the following file opened by Git.

The ‘~3’ means three commits before HEAD.

```
1 pick a958dff # continued ct2->lab 05
2 pick 52c15fd # elaborated root readme for ct1
3 pick 555cf90 # moved and extended 'git-introduction.md'
4
5 # Rebase 4abb7c9..555cf90 onto 4abb7c9 (3 commands)
6 #
7 # Commands:
8 # p, pick <commit> = use commit
9 # r, reword <commit> = use commit, but edit the commit message
10 # e, edit <commit> = use commit, but stop for amending
11 # s, squash <commit> = use commit, but meld into previous
12 #
13 # f, fixup [-C | -c] <commit> = like "squash" but keep only the
14 #
15 #
16 #
17 #
18 # x, exec <command> = run command (the rest of the line) using
19 #
20 # b, break = stop here (continue rebase later with 'git rebase
21 #
22 # d, drop <commit> = remove commit
23 # l, label <label> = label current HEAD with a name
24 # t, reset <label> = reset HEAD to a label
25 # m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
26 #
27 #
28 #
29 #
30 #
31 # u, update-ref <ref> = track a placeholder for the <ref> to
32 #
```

```
33 #             commits. The <ref> is updated at the end
34 #             of the rebase.
35 #
36 # These lines can be re-ordered; they are executed from top
37 # to bottom.
38 #
39 # If you remove a line here THAT COMMIT WILL BE LOST.
40 #
41 # However, if you remove everything, the rebase will be
42 # aborted.
```

The commits are listed oldest to youngest from the top. I could reorder them by just moving the lines. Or remove a commit by changing the ‘pick’ at start of its line to ‘drop’.

I will not go into detail on the available actions, just one note on ‘squash’ (combine), which is a function I use regularly. Commits are always squashed into their parents (upwards in this list).

After editing the list as intended, save and close the file. If you chose actions, which need further input, Git will open another file. If you for example chose to reword a commit, Git will open a file with the old commit message, which you can edit, save and close.

Many functions require a force push. **Again be very careful with force pushes.**

Troubleshooting

I lost a commit!

Do not worry, use `git reflog`. The reference log keeps track of references and usually remembers “dangling” commits. This allows you to checkout or cherry-pick a “lost” commit or reference.

I tried to pull, but got a merge conflict?!

That is why you should always fetch first and pull only, after checking for conflicts.

This problem can occur, when someone rewrote the history and your local HEAD points to a commit that does no longer exist.

The solution is very easy: `git reset --hard <remote-name>/<branch-or-reference-name>`

If you have local commits that conflict with commits on the remote, see [Merging](#) or [Rebasing](#).

Further reading

Here a few functions this guide does not cover, for you to read up on, if interested:

- `git stash` allows to stash local changes, enabling actions like pull or rebase.
- `git blame` can determine, who is responsible for code changes (line by line).
- `git cherry-pick` allows to add single (or several) commits from a feature branch to the current branch, without having to add the entire feature branch.
- `git bisect` uses binary search to determine the commit, which introduced a bug.
- **command aliases** let you access often used command (with flags) via `git <alias>`. (This is distinct from the shells aliases.)

Glossary

Term	Description
branch	Separate named reference originating from an older commit on the source branch. A branch can originate from main/master (the trunk in the tree analogy) or any other branch.
commit	Repository state defined as delta to its parent. Each commit is identified by its commit hash, a 40 byte long string of alphanumeric characters. The hash can be used in many Git commands, e.g. <code>diff</code> , <code>reset</code> or <code>checkout</code> . A commit usually has one parent, but can have an arbitrary number of children, one for each branch originating from the commit.
commit reference	Named pointer to a specific commit. An example would be <code>HEAD</code> , which points to the currently checked out commit. Branches and tags are references too.
head	Pointer to commit currently checked out. On remote it will usually point to <i>main</i> . With <code>git checkout</code> one can checkout a to e.g. test something or make new branch. In such a case the head would point to that commit.

Term	Description
index / staging area	<code>git add</code> puts file/directories (rather the changes made to them) into a cache, called <i>index</i> or <i>staging area</i> (in arguments it is also called chache). <code>git commit</code> adds the changes in that cache to the history with a hash, a descriptoin and a few more information about the commit.
working tree	Local copy of the repository. a working tree is clean if it has no uncommitted changes.
